JOHANNES KEPLER
UNIVERSITY LINZ

Simon Grundner
k12136610

Institute for Integrated Circuits and Quantum Computing

k12136610@students.jku.at
https://github.com/s-grundner/LVA-EIC-KV

January 21, 2026

Tiny Tapeout - MIDI Polyphonic Synthesizer

Technical Report

KV Integrated Circuits Design - WiSe25

JOHANNES KEPLER UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Abstract MIDI Polyphonic Synthesizer on the Tiny Tapeout ASIC

Contents

Abstract

The project on hand implements a polyphonic audio square wave synthesizer in Verilog, which can be controlled via the musical instrument digital interface (MIDI) protocol. The project is allocated on the TTSKY25b shuttle of the TinyTapeout project and is to be manufactured under the SkyWater 130 nm process. This report documents functionality as well as implementation and discusses design constraints, imposed by the limited physical space on the shuttle.

Acronyms

ASIC application specific integrated circuit

CMOS Complementary Metal Oxide Semiconductor

DAC digital to analog converter

DAW digital audio workstation

FSM finite statemachine

HDL hardware description language

IC Integrated Circuit

GM1 general MIDI level 1

GM2 general MIDI level 2

LUT lookup table

MIDI musical instrument digital interface

MSB most significant bit

PCB printed circuit board

PDK process development kit

PWM pulse width modulation

UART universal asynchronous reciever transmitter

USB universal serial bus

SPN scientific pitch notation

12ET twelve tone equal temperament

I2S inter-integrated sound

1. Introduction

MIDI is a versatile digital interface, which packs note expressions into a simple protocol. The note expression often origins from a MIDI-controller (source) and contains information about which key on the pianoroll is affected and how it is affected. A key thing to notice is, that the note expression itself does not contain any information about how the audiosignal sounds. The synthesis of the waveform is handled by a MIDI-instrument (sink) and translates the note expression into a audio timesignal. Figure 1 shows how a MIDI-keyboard might interface with a MIDI compatible synthesizer.

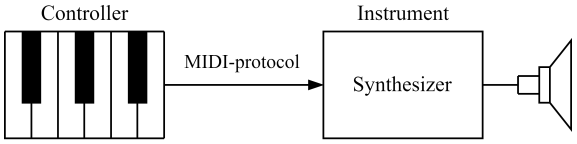

Figure 1: Setup of a controller-instrument-pair communicating via the MIDI-protocol.

The project implements such a MIDI-instrument and converts incoming MIDI-signals into audible squarewaves. The project is written in the hardware description language (HDL) Verilog specifically to be built as a digital application specific integrated circuit (ASIC) design by the open source LibreLane toolchain. The resulting chip will be taped out in collaboration with the Tiny-Tapeout project.

The Tiny-Tapeout project makes manufacturing ASIC-designs for individuals feasable by combining multiple smaller designs on a single chip under the usage of an open source process development kit (PDK). The chip space is sectioned in tiles, where each tile has around $160 \times 100\,\mu m$ of available space. The project on hand occupies one tile. The active run at this time, TTSKY25b, manufactures the chip under SkyWater 130 nm PDK ruleset.

2. Digital Instruments and Music Theory

Before discussing the design, a few fundamentals of music theory have to be considered and how implementing a digital instrument in silicon is sensible. The MIDI Manufactures Association defines the general MIDI level 1 (GM1) and its extension the general MIDI level 2 (GM2) standard to provide guidelines for developing MIDI compatible instruments.

2.1 The MIDI-Protocol

The MIDI-protocol is based on a serial asynchronous data transmission, where at most three bytes are sent in succession, with a dataformat as shown in Table 1. It is noted, that not all commands require a second data byte which is why it is specified in brackets as an optional byte. In this report all required commands provide a second data byte, therefor a length of three bytes will be assumed from now on.

Table 1: Dataformat of a MIDI-protocol transmission

Byte 1		Byte 2	Byte 3
Statusbyte		Databyte 1	[Databyte 2]
Command	Channel		

Table 2: Excerpt from the MIDI 1.0 specification message summary for channel voice messages [5].
$\mathsf{N} = \mathtt{nnnn} \in [0;15]$ maps to a MIDI channel number 1-16.
$\mathsf{K} = \mathtt{kkkkkkk} \in [0;127]$ is the note number. (e.g. which key on a piano is pressed/released)
$\mathsf{V} = \mathtt{vvvvvvv} \in [0;127]$ is the velocity (how fast a note is pressed/released)

Status	Databytes	Description
`1000nnnn`	`0kkkkkkk` `0vvvvvvv`	Note Off event. This message is sent when a note is released (ended).
`1001nnnn`	`0kkkkkkk` `0vvvvvvv`	Note On event. This message is sent when a note is pressed (started).

The first byte combines two four-bit values representing a command and a channel number. Depending on the command code, the following databytes can be interpreted accordingly by a MIDI-sink listening on a pre-set channel. The most important and only command codes used in this project are listed in Table 2. A seven bit note number (denoted as `kkkkkkk` in Table 2) maps to the piano keys of a twelve tone equal temperament (12ET), where the middle C (C_4) is equivalent to a MIDI note number of 60 in decimal [4, sec 2.7.1.1].

2.2 Instrument Tuning

A musical octave is an interval of two notes, where the frequency of the higher note is exactly twice as high as the lower note. The 12ET is a system which divides this octave into twelve notes, which are referred to in scientific pitch notation (SPN) as letters from A to G with accidentals (♭, ♯) and an octave number n as shown in Table 3.

Table 3: SPN Nomenclature for notes in a 12 tone tempered octave, where n is the octave-number.

B_{n-1}	C_n	$\frac{C\sharp_n}{D\flat_n}$	D_n	$\frac{D\sharp_n}{E\flat_n}$	E_n	F_n	$\frac{F\sharp_n}{G\flat_n}$	G_n	$\frac{G\sharp_n}{A\flat_n}$	A_n	$\frac{A\sharp_n}{B\flat_n}$	B_n	C_{n+1}

These notes are equally tempered, meaning the frequency-ratio of two adjacent notes is constant for all musical notes. For an octave of 12 notes, this ratio equates to $\sqrt[12]{2} \approx 1.05946$, leading naturally to a logarithmic scale.

To assert actual frequency values to the notes, a pitch standard has to be decided on. GM2 suggests the ISO-16 pitch standard, where note number 69 in decimal (A_4) is tuned to 440 Hz [4, sec. 2.7.1.1]. With this reference frequency, the absolute frequency of all notes can be calculated by multiplying or dividing by the defined ratio, yielding Equation 1 for the note frequency $f_{note}(K)$, where K is the MIDI note number.

$$f_{note}(K) = 440\,\text{Hz} \cdot (\sqrt[12]{2})^{(K-69)} = 440\,\text{Hz} \cdot 2^{(K-69)/12} \tag{1}$$

Theoretically, the MIDI note could use all 7 bits, so a note range from $[0; 127]$. As MIDI is heavily used with piano style controllers, the maximum relevant range a MIDI intrument must generate audio is that of a 88-key piano, ranging from note numbers 21 to 108 [3, p. 10][4, p. 25]. This yields a total of 108 notes.

2.3 Instrument Characteristics

The frequency in Equation 1 yields the fundamental frequency of the sound oscillation generated by the instrument. However, the characteristic sound an instrument produces is caused by overtones relative to its fundamental. For example, the Grand Piano [1] in the digital audio workstation (DAW) Ableton Live (see Appendix A) is characterised by overtones as seen in Figure 2.

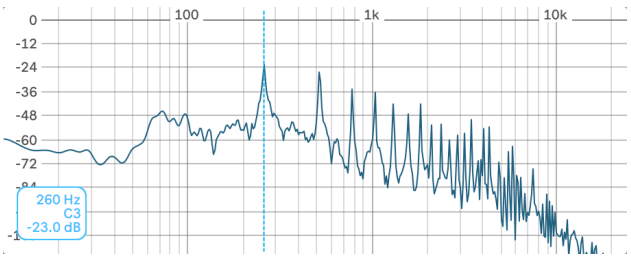

Figure 2: Overtones of a grand piano instrument playing a middle C (C3 in Lives notation).
Abscissa: Audiolevel in dB relative to the fullscale range of the audio engine.
Ortinate: Frequency in Hz.

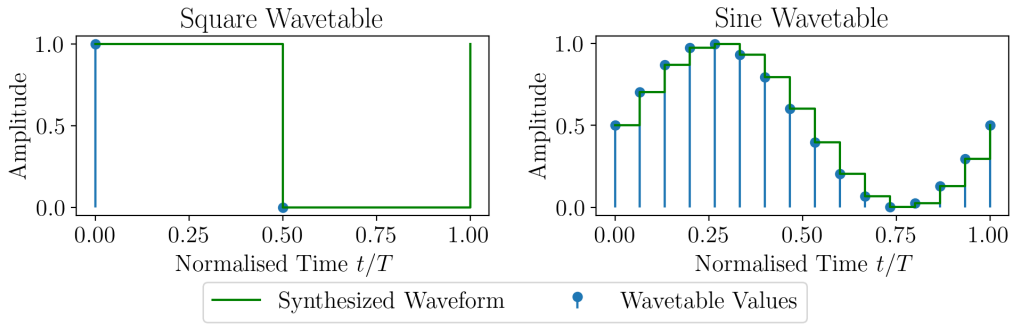

Figure 3: Wavetables for a 1-bit squarewave (left) and a 16-bit sinewave (right)

A classical method to characterise the voice of a synthesizer is by using a wavetable. A wavetable is a discrete list of values representing the waveform amplitude over the span of one period (Figure 3). To synthesize the audio waveform, the table is being iterated at a certain speed, such that one loop spans over the period of the fundamental frequency.

To implement this in the design, the output capabilites of the ASIC have to be considered. As the design is purely digital, the output levels of the chip can only assume the logic levels high and low, which leaves following options:

1. Feed an audio stream to an external DAC using serial connectivity such as I2S.
2. Use all output pins as a parallel interface to drive a parallel DAC asynchronously.
3. Have the digital output be the synthesized waveform.

For this design, the latter option is chosen, which has several benefits. Using a single digital output directly as the audio waveform is equivalent to a 1-bit wavetable, resulting in a squarewave. For a 1-bit wavetable the output can simply be toggled every half period.

The spectrum of an ideal squarewave with infinite runtime is defined by a fourier series in Equation 2 containing odd multiples

$$\omega_n = 2\pi n T^{-1}, \qquad n = 1, 3, 5, \ldots$$

of the fundamental frequency ω_1. The amplitude for harmonics of higher order declines hyperbolically [6], leading to a spectrum as shown in Figure 4.

$$s(t) = \sum_{n=1,3,5,\ldots}^{\infty} \frac{4}{n\pi} \sin(\omega_n t) \quad \overset{\|\cdot\|}{\circ\!\!-\!\!-} \quad \|S(j\omega)\|_{\omega>0} = \sum_{n=1,3,5,\ldots}^{\infty} \frac{4}{n} \delta(\omega - \omega_n) \tag{2}$$

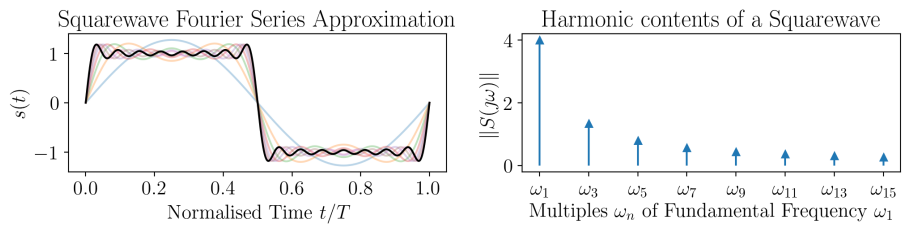

Figure 4: Approximation of a squarewave by adding odd harmonics one by one

2.4 Polyphony

It is common for instruments to play two notes simultaneously, which is caused by receiving multiple note-on commands without turning already active MIDI notes off. An important parameter is the number of voices the instrument can produce. An instrument is considered polyphonic, if the number of voices is at least two. In this case, each voice is represented by a individual squarewave oscillators. One of the main challenges in this design is to reduce the area for the implementation of a voice oscillator to achieve a high polyphony count.

The audiosignal is directly sent to a digital pin. However, overlapping squarewaves of differing phase and frequency, again produce an analog signal (shown in Figure 5), which cannot be output by a single pin. Therefor, each voice has its own output pin. The individual voices are to be summed together by external circuitry, for example by an operational amplifier circuit.

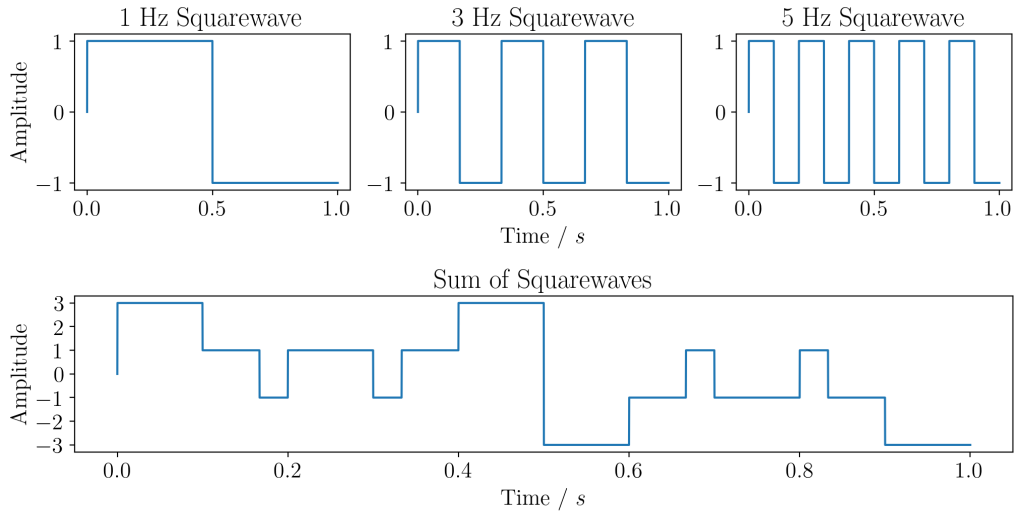

Figure 5: Individual Squarewaves summed up to an analog signal

2.5 MIDI Physical Layer

The MIDI protocol was originally transmitted asynchronously over a differential pair, which was daisychained between MIDI devices using 5 pole DIN connectors. This interface required additional hardware before the connection to a UART line driver [3, p. 2]. With rising popularity of the universal serial bus (USB), MIDI devices adapted to this standard. Nowadays most MIDI devices are able to communicate via USB to a host device. Therefor MIDI instruments can also be controlled via USB under the usage of a USB to UART serial interface as it is intended with this design.

2.5.1 UART

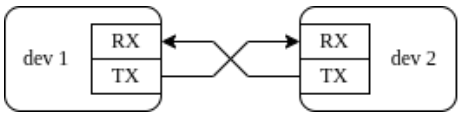

Figure 6: Wiring of two devices communicating via a UART connection

The universal asynchronous reciever transmitter or UART is a simple interface which allows two devices to communicate over a serial connection. The devices are connected as shown in Figure 6. Each device can initiate a communication on its own behalf, without the acknowledgement of the other device, allowing for simultaneously sending and receiving data. The connection does not feature a clock line, which indicates an asynchronous communication. The timing for the period of individual bits is determined by a baudrate, which has to be specified in both devices. For MIDI devices, the standard baudrate is $31\,250\,\mathrm{bit\,s^{-1}}$ [3].

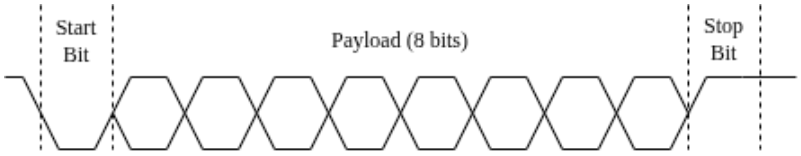

Figure 7: Dataframe of a UART communication

In an idle state, the dataline is normally set to a logic level *high* by the `TX` output. To initiate a data transfer, the transmitter pulls the communication line low for one bit-period ($\frac{1}{\text{baudrate}}$). Then, eight bits are sent with the specified baudrate followed by a stop bit, which pulls the communication line high again. A complete datatransmission is shown in Figure 7.

Using a UART connection is very practical for transmitting MIDI signals between a controller and an instrument, as the serial MIDI-Data is compatible with the UART-payload size and does not require additional wires for e.g. clock or select signals.

3. System Overview

The source code can be found under the follwing link:
https://github.com/s-grundner/LVA-EIC-KV

3.1 Pinout and Top Level Module

The project features four voices implemented as individual squarewaves oscillators. The output of each oscillator is routed to the outputpins `uo[0]`…`uo[3]`. Output `uo[7]` yields a PWM Signal, which encodes the number of actively running oscillators. This output may be used to control the gain of external circuitry when mixing the oscillator outputs together. The project has one input `ui[3]` to which the MIDI signal has to be supplied to. This pin is also connected to the UART-TX output of the RP2040 Microcrontroller, which is populated on the TinyTapeout test PCB. The RP2040 may be used to emulate a MIDI controller or to interface the instrument via USB. Figure 8 summarises the in- and outputs of the ASIC.

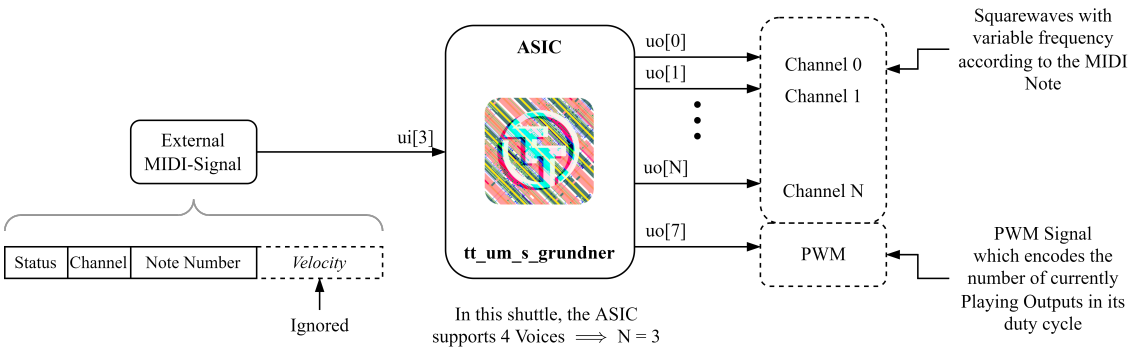

Figure 8: Input and output visualisation and description

3.2 Block Diagram

An abstract overview of the project implementation can be seen in Figure 9. The block-titles outside of each block repesent the Verilog module name coherent with the code-repository.

The signal flow starts at the input, where the MIDI messages are received via UART. The bits of the input stream are collected into bytes, which are then fed into a MIDI decoder. The MIDI decoder extracts the note- and channel number and if an on or off command is received.

The note number indexes a lookup table (LUT), which returns a counterbase for the corresponding frequency. Each oscillator in the stack is listening on its own MIDI-channel. The note-on/note-off commands only affect the oscillator listening on this channel. Upon recieving a note on command, the counterbase value at the input of an oscillator is latched

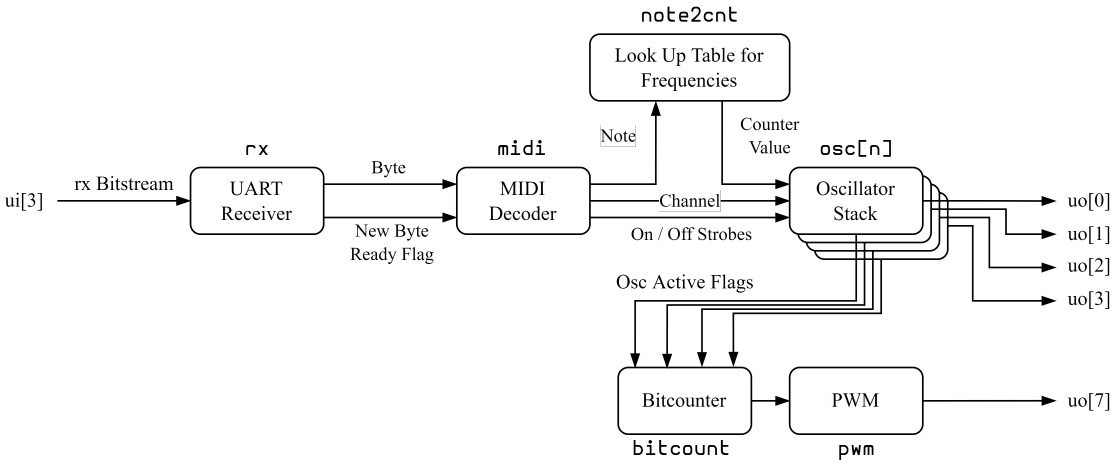

Figure 9: Abstract Blockdiagram of the Synthesizer (`synth` module)

and a counter starts counting repeatedly in this time-base. The output of the oscillator is then toggled at each counter overflow, which generates the squarewave output.

3.3 Layout

To demonstrate the density of the layout, an image of the generated GDS is shown in Figure 10.

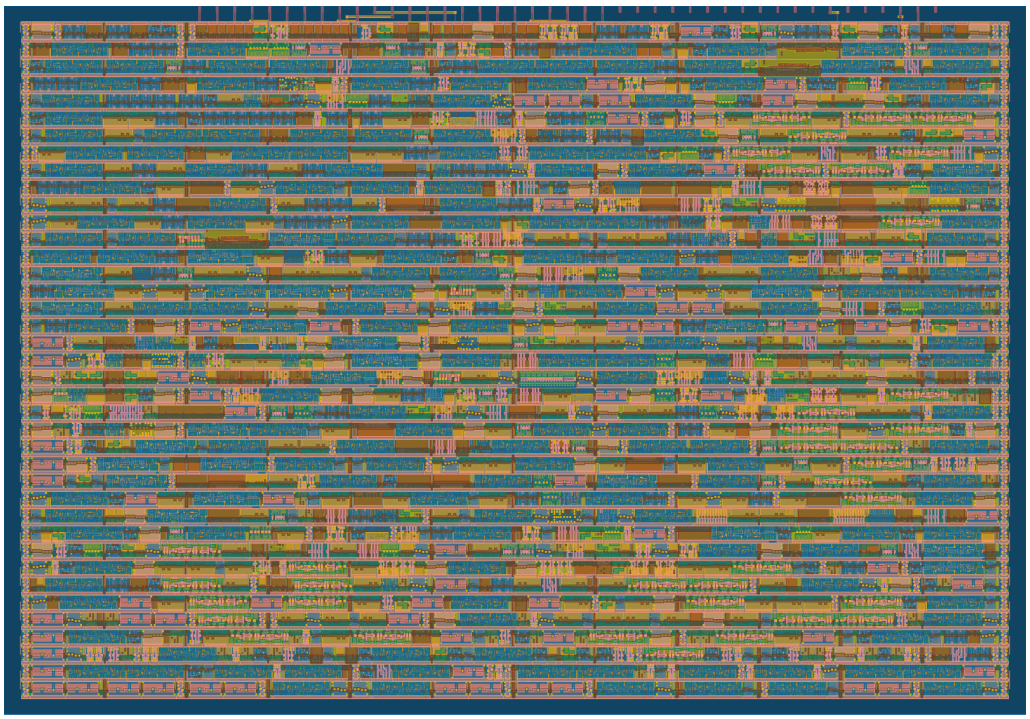

Figure 10: Layout of the Tile

The routing statistics generated by the TinyTapeout pipeline are listed in Table 4. Especially a space utilisation of 74.275 % shows, that space-optimization was crucial. Per

default, the maximum density is set to 60 %, which had to be adjusted to fit more features. The documentation of the TinyTapeout project stated, that a density of up to 80 % worked in previous shuttles without issues. The layout as it is shown in Figure 10 is comprised of logic cells which are predefined by the PDK. Which logic cells are generated from the Verilog code can be seen in Table 5.

Table 4: Routing statistics

Utilisation / %	Wire length / µm
74.275	23342

Table 5: Used Logic cells of the PDK and the number of occurences

Category	Cells	Count
Fill	decap fill	733
Combo Logic	a22o a41o and2b a2bb2o o22ai a31o a22oi o21ai a221o o22a o2111ai a211o or4b a21oi a21o o211a and3b a311o o21a o2bb2a o221a a2111o and4b o211ai or4bb a211oi or3b a31oi o41ai o21ba a21bo a32o a221oi o2111a and4bb o32a	228
Tap	tapvpwrvgnd	225
Flip Flops	dfrtp dfstp	191
Multiplexer	mux4 mux2	180
Misc	conb dlymetal6s2s dlygate4sd3	98
AND	and3 and2 and4 a21boi	95
Buffer	clkbuf buf	79
NOR	nor2 xnor2	59
Inverter	inv	49
OR	or2 xor2 or3 or4	48
NAND	nand2 nand3 nand2b	32
Clock	clkinv	10
1069 total cells (excluding fill and tap cells)		

3.4 Global Settings

Certain parameters are reused throughout the entire project. These parameters are collected into a `global.v` file, which is included in every Verilog source file. The defined global variables are listed in Table 6. How these parameters are chosen, will be documented throughout the report.

Table 6: Content of `global.v` in a tabular format

Variable	Value	Description
`F_CLK_HZ`	`3_496_503`	Global Clock Speed which has to be supplied to the IC
`F_CLK_PERIOD_NS`	`1_000_000_000 / `F_CLK_HZ`	
`F_BAUD`	`31250`	
`F_BAUD_PERIOD_NS`	`1_000_000_000 / `F_BAUD`	
`MIDI_PAYLOAD_BITS`	`8`	
`MIDI_NOTE_BW`	`7`	
`OSC_VOICES`	`4`	MAX 7
`OSC_VOICES_BW`	`$clog2(`OSC_VOICES+1)`	
`OSC_CNT_BW`	`16`	Counter bit width
`OSC_ROM_BW`	`7`	Note ROM bit width

3.5 Modules

A quick overview and a brief description on the Verilog modules is given in Table 7. How the modules are connected can be seen in Figure 9.

Table 7: Overview of Verilog modules

Module	Description
`tt_um_s_grundner`	Tiny Tapeout top level module
`synth`	Combines reciever, lookup table und oscillator stack
`osc`	generates a squarewave of specified frequency upon receiving a note on command. Disables output for note off commands
`midi`	Extracts note on/off commands, channel and note number from incoming bytes.
`rx`	RX frontend of midi input. 31250 baud, 8 data bits, no parity
`note2cnt`	Convert note numbers to counter base via LUT
`counter`	Counts up on each clock cycle
`bitcount`	Returns the number of high bits in the input vector
`pwm`	PWM output for encoding the number of active Oscillators. The Frequency of the PWM output is: $f = f_{CLK}/N_{VOICES}$

4. Receiver

As UART is a common interface, there are already several implementations available in Verilog. For this project, an open source implementation[1] under the MIT license was used as a starting point and has then been adjusted to the project. As the project only recieves data and does not sent any, only the UART-receiver has to be implemented.

Listing 1: Receiver module ports

```
module rx (
    input wire clk_i,
    input wire nrst_i,
    input wire rxData_i,
    output reg dataReady_o,
    output reg [`MIDI_PAYLOAD_BITS-1:0] midiData_o
);
```

4.1 Statemachine

At the core of the receiver is a finite statemachine (FSM) (Figure 11), with the four states a UART transmission can assume: `IDLE`, `START`, `RECV` and `STOP`. A combinatorial process decides, which state is registered with the next clock cycle as shown in Listing 2.

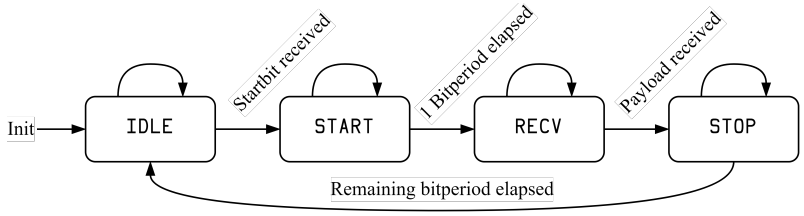

Figure 11: Statemachine diagram for the receiver

Listing 2: Combinatorics to select next FSM state

```
    // Next State Conditions
    wire nextBitReady = (cycleCounter == COUNT_REG_LEN'(CYCLES_PER_BIT)) ||
                        (fsmState == FSM_STOP) &&
                        (cycleCounter == COUNT_REG_LEN'(CYCLES_PER_BIT_HALF));
    wire payloadDone = (bitCounter == `MIDI_PAYLOAD_BITS);

    // Select Next State
    always @(*) begin : nextFsmState_p
        case (fsmState)
            FSM_IDLE:  nextFsmState = rxDataReg0 ? FSM_IDLE : FSM_START;
            FSM_START: nextFsmState = nextBitReady ? FSM_RECV : FSM_START;
            FSM_RECV:  nextFsmState = payloadDone ? FSM_STOP : FSM_RECV;
            FSM_STOP:  nextFsmState = nextBitReady ? FSM_IDLE : FSM_STOP;
            default:   nextFsmState = FSM_IDLE;
        endcase
    end
```

[1] https://github.com/ben-marshall/uart

1. **IDLE**: With each clock cycle, the input is latched in `rxDataReg` (Listing 3). The data register is normally high in the idle state. When a startbit is detected - so the `rxDataReg` wire assumes a low state - the next state is set to `START`.
2. **START**: The counter `cycleCounter` divides the frequency into the baudrate defined in `global.v` (Listing 4). After the period of the start bit has elapsed, the next state is set to `RECV`.
3. **RECV**: A second counter `bitCounter` counts the number of sampled databits at the input (Listing 5). When the expected payload is recieved, the next state is set to `STOP`.
4. **STOP**: The cycle counter counts the remaining half bit period and sets the next state to `IDLE`.

Listing 3: Register the input wire with each clock cycle. To break up long combinatorial chains before registering, the received data is internally latched a second time.

```
1     always @(posedge clk_i or negedge nrst_i) begin : rxBuffer_p
2         if (!nrst_i) begin
3             rxDataReg0 <= 1'b1;
4             rxDataReg1 <= 1'b1;
5         end else begin
6             rxDataReg0 <= rxDataReg1;
7             rxDataReg1 <= rxData_i;
8         end
9     end
```

Listing 4: Increments the cycle counter. The counter is incremented in each clockcycle, when not in the `IDLE` state. The counter is reset after the counter reaches the maximum cycles per bit count. After the counter overflows, a bit period defined by the baudrate is elapsed.

```
1     always @(posedge clk_i or negedge nrst_i) begin : countCycles_p
2         if (!nrst_i) begin
3             cycleCounter <= COUNT_REG_LEN'(0);
4         end else if (nextBitReady) begin
5             cycleCounter <= COUNT_REG_LEN'(0);
6         end else if (fsmState == FSM_START || fsmState == FSM_RECV || fsmState ==
      FSM_STOP) begin
7             cycleCounter <= cycleCounter + COUNT_REG_LEN'(1);
8         end
9     end
```

Listing 5: Increments the bit counter. The counter increments, when the cycle counter reaches its maximum (a new bit is ready to be sampled) and the FSM is in the `RECV` state. The counter is reset when the FSM leaves the `RECV` state.

```
1     always @(posedge clk_i or negedge nrst_i) begin : countBit_p
2         if (!nrst_i) begin
3             bitCounter <= 4'b0;
4         end else if (fsmState != FSM_RECV) begin
5             bitCounter <= 4'b0;
6         end else if (fsmState == FSM_RECV && nextBitReady) begin
7             bitCounter <= bitCounter + 4'b1;
8         end
9     end
```

To acquire the data, the input is latched into a sample register in the middle of a bit period (Listing 6). When the bit period has elapsed, the sampled bit is then collected in the `midiData` byte (Listing 7), which is then assigned to the output of the module. Whenever the whole payload is received, the `dataReady_o` strobe[2] is pulsed (Listing 8), which is required by the MIDI decoder.

Listing 6: Sample a bit in the middle of a bit period.

```
1     always @(posedge clk_i or negedge nrst_i) begin : sampleBit_p
2         if (!nrst_i) begin
3             sampledBit <= 1'b0;
4         end else if (cycleCounter == COUNT_REG_LEN'(CYCLES_PER_BIT_HALF)) begin
5             // Take sample in the middle of a bit
6             sampledBit <= rxDataReg0;
7         end
8     end
```

Listing 7: Collect sampled data into a MIDI byte. The latest bit is always inserted at the MSB position. Already received bits are shifted right.

```
1     integer i;
2     always @(posedge clk_i or negedge nrst_i) begin : updRxBuffer_p
3         if (!nrst_i) begin
4             midiData <= `MIDI_PAYLOAD_BITS'b0;
5         end else if (fsmState == FSM_IDLE) begin
6             midiData <= `MIDI_PAYLOAD_BITS'b0;
7         end else if (fsmState == FSM_RECV && nextBitReady) begin
8             midiData[`MIDI_PAYLOAD_BITS-1] <= sampledBit;
9             for (i = `MIDI_PAYLOAD_BITS - 2; i >= 0; i = i - 1) begin
10                midiData[i] <= midiData[i+1];
11            end
12        end
13    end
```

Listing 8: Output a data ready strobe whenever a transmission is complete.

```
1     always @(posedge clk_i or negedge nrst_i) begin : dataReady_p
2         if (!nrst_i) begin
3             dataReady_o <= 1'b0;
4         end else if (fsmState == FSM_STOP) begin
5             dataReady_o <= payloadDone;
6         end else begin
7             dataReady_o <= 1'b0;
8         end
9     end
```

[2] A strobe is a signal, which is high until the next rising edge of the clock signal.

5. MIDI-Decoder

The MIDI decoder is responsible for extracting MIDI command and channel aswell as the note number from the received bytes. The in- and outputs of this module can be seen in Listing 9.

Listing 9: MIDI module ports.

```
module midi (
    input wire clk_i,
    input wire nrst_i,
    input wire midiByteValid_i,
    input wire [`MIDI_PAYLOAD_BITS-1:0] midiByte_i,
    output reg [`OSC_VOICES_BW-1:0] ch_o,
    output reg [`MIDI_NOTE_BW-1:0] note_o,
    output reg noteOnStrb_o,
    output reg noteOffStrb_o
);
```

5.1 Statemachine

Here again, a finite statemachine shown in Figure 12 underlies the MIDI decoder. The decoder has four states, representing which type of byte is currently received. `IDLE`, `CMD` and `NOTE` (Listing 10). The FSM state switches states upon receiving a data ready strobe (`midiByteValid_i`) from the UART reciever. The input byte is also latched on this event (Listing 11) and is then being processed according to the state.

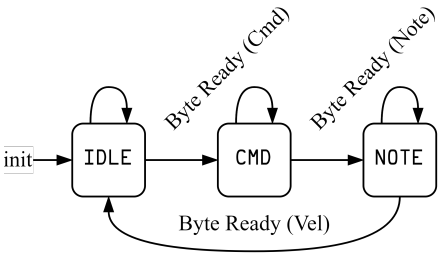

Figure 12: MIDI decoder statemachine

Listing 10: States of the finite statemachine. The next state will be immediately set, the assignment condition of the next state to the current state is handled in a register process.

```
    always @(*) begin : nextFSM_p
        case (fsmState)
            FSM_IDLE: nextFsmState = FSM_CMD;
            FSM_CMD: nextFsmState = FSM_NOTE;
            FSM_NOTE: nextFsmState = FSM_IDLE;
            default: nextFsmState = FSM_IDLE;
        endcase
    end
```

The velocity data itself is ignored, however, it still needs to be present at the input, because the data ready strobe of the last byte is needed to switch into the `IDLE` state. For most instruments, the velocity is interpreted as the loudness of the tone. As the digital output only has one amplitude level, this value is obsolete.

Listing 11: Register the next FSM state aswell as the input byte, when the input byte is valid.

```
    always @(posedge clk_i or negedge nrst_i) begin : midiByteReg_p
        if (!nrst_i) begin
            midiByteReg <= {`MIDI_PAYLOAD_BITS{1'b0}};
            fsmState <= FSM_IDLE;
        end else if (midiByteValid_i) begin
            midiByteReg <= midiByte_i;
            fsmState <= nextFsmState;
        end
    end
```

1. **IDLE**: The FSM resides in the idle state, until the first byte is ready. On this event, the state is switched to `CMD`.
2. **CMD**: The received byte is split into the channel number and the command in Listing 12 according to Table 1. When the next byte is available, the state switches to `NOTE`.
3. **NOTE**: The previously received command is now available for evaluation. The output strobes `noteOnStrb_o` and `noteOffStrb_o` are assigned accoring to the command in Listing 13 and will notify the oscillatorstack, if the current channel output needs to be turned on or off. The received byte is assigned to the note output in Listing 14.

Listing 12: Deconstruct the status byte into channel number (upper 4 bits) and command (lower 4 bits). Forwards the channel to the output and latches the command internally to be used by the next state.

```
    always @(posedge clk_i or negedge nrst_i) begin : statusEval_p
        if (!nrst_i) begin
            cmd <= {CMD_BW{1'b0}};
            ch_o <= {`OSC_VOICES_BW{1'b0}};
        end else if (fsmState == FSM_CMD) begin
            cmd <= midiByteReg[7:4];
            ch_o <= midiByteReg[`OSC_VOICES_BW-1:0];
        end
    end
```

Listing 13: Forwards the MIDI byte ready strobe to either the note-on or note-off strobe, according to the received command.

```
    always @(posedge clk_i or negedge nrst_i) begin : noteOnOffStrb_p
        if (!nrst_i) begin
            noteOnStrb_o <= 1'b0;
            noteOffStrb_o <= 1'b0;
        end else if (fsmState == FSM_NOTE && midiByteValid_i) begin
            noteOnStrb_o <= (cmd == CMD_NOTE_ON) ? 1'b1 : 1'b0;
            noteOffStrb_o <= (cmd == CMD_NOTE_OFF) ? 1'b1 : 1'b0;
        end else begin
            noteOnStrb_o <= 1'b0;
            noteOffStrb_o <= 1'b0;
        end
    end
```

Listing 14: The received byte is forwarded to the note output of the module.

```
1     always @(posedge clk_i or negedge nrst_i) begin : note_p
2         if (!nrst_i) begin
3             note_o <= {`OSC_ROM_BW{1'b0}};
4         end else if (fsmState == FSM_NOTE) begin
5             note_o <= midiByteReg[`MIDI_NOTE_BW-1:0];
6         end
7     end
```

5.2 Limitations

According to the GM1 specifications, a sound generating instrument must implement a set of features to ensure compatibility with various controller outputs / MIDI-sequences [2]. This project does not fully comply with GM1 as even the smallest feature set requires a much more complex MIDI statemachine, which is beyond the goals of this project. Because of this, directly connecting a MIDI-controller to the IC might lead to an invalid state of the MIDI-state machine, as not all MIDI-Messages are guaranteed to be 3 bytes long. Edge cases, which are required to be handled by the device, are evaluated in section 8.

A particular feature of the MIDI decoder, which is normally not found in typical applications, is that polyphony is only achieved by sending parallel inputs onto different MIDI channels. To clarify this, an example for two input scenarios is given:

A: A key is held down, sending a note-on command on channel 1. The key is synthesized by the IC, outputting a squarewave on pin `uo[0]`. Upon pressing another key, which is also sent on channel 1, the IC overrides the output 1, playing the most recent note on the channel. This is even the case when the remaining oscillators are in the idle state.

B: A key is held down, [...]. Upon pressing a key, which is sent onto a different channel than 1, e.g. channel 2, the output `uo[0]` continues to play the old note and the newly pressed note is being sent onto `uo[1]`. This is the way, in which polyphony is achieved.

This behaviour is called *voice stealing*. Voice stealing is implemented by instruments, so that if the number of keys pressed by the controller exceeds the polyphonic count of the instrument, the instrument switches the earliest played note off and allows the newly pressed key to play a note. This static channel routing approach was necessary to reduce the logic of the MIDI decoder.

6. Synthesizer

The synthesizer is the top level module of the design which takes the note data, converts it into a time base and routes it to the oscillators. Other than connecting all the individual modules (see Figure 9), the synth module is responsible for automatically generating an array of oscillator instances (oscillator stack) via a *generate-for* demonstrated in Figure 13. This approach allows for adjusting the number of voices with a single parameter `OSC_VOICES` in the `global.v` file. Should further optimizations be found, the parameter can be changed to increase polyphony count. This automatically opens up additional MIDI channels.

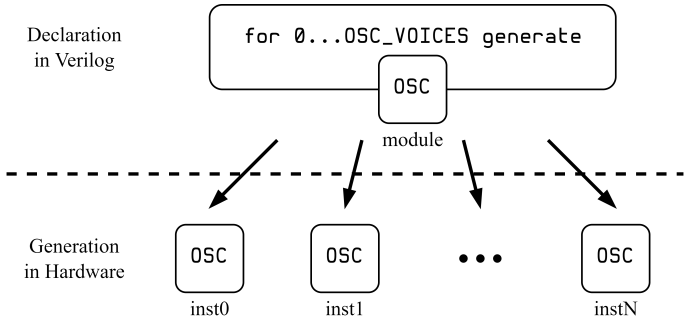

Figure 13: Generate multiple instances of a single module under the use of a the generate feature in Verilog.

All oscillators share the same inputs. Each oscillator decides internally, if the input is processed when the channel number matches. The generation and wiring of the oscillator stack can be seen in Listing 15.

Listing 15: generate-for implementation to create an array of oscillators for the required polyphonic count.

```
1     genvar i;
2     generate
3         for (i = 0; i < `OSC_VOICES; i = i + 1) begin : oscStack_gen
4             // Truncate to reduce operation bitwidth of channel comparison
5             localparam OSC_CH = i[`OSC_VOICES_BW-1:0];
6             /* verilator lint_off WIDTHTRUNC */
7             // OK: Warning only occurs if OSC_VOICES is a power of 2
8             osc osc_inst (
9                 .clk_i(clk_i),
10                .nrst_i(nrst_i),
11                .baseCntPeriod_i(oscCmp),
12                .shift_i(oscShift),
13                .ch_i(channel == OSC_CH),
14                .active_o(activeOscs[OSC_CH]),
15                .noteOnStrb_i(noteOnStrb),
16                .noteOffStrb_i(noteOffStrb),
17                .wave_o(oscOut_o[OSC_CH])
18            );
19            /* verilator lint_off WIDTHTRUNC */
20        end
21    endgenerate
```

6.1 Timebase Calculation

The MIDI decoder yields just the note number, which can not yet be interpreted as a frequency. For this, a lookup table is implemented serving, as an interface between note number and timebase.

As seen in Equation 1, the process to convert a note number to a frequency, let alone a counter value, is not a straight forward calculation to be implemented in CMOS logic. Multiple challenges arise with this calculation, especially for divisions and modulo operations. The only convenience is, that the base of the exponential is two. For this, logical shift operations («,») will be leveraged.

Instead of performing the calculation at all, a lookup table could be created and indexed with the note number directly, which returns a counter value. With this approach, the space which is occupied just by the lookup table has to be considered. Storing all the notes over the MIDI piano note range (21-108) would require a total of 88 entries, each with the bitwidth of the oscillators resolution.

To efficiently calculate the note frequencies, a LUT is used to store the half[3] counter periods of only a single octave. The other notes can then be reached, by dividing (») or multiplying («) by 2.

6.1.1 Choosing a Clock Frequency

To represent a time duration with a counter, a core frequency needs to be known. Only then a time resolution can be defined, which is represented by a counter increment.

From Figure 14 can be derived, that the relation between the half period counter value N and the clock frequency is given by

$$\mathsf{N}(\mathsf{K}_{\mathrm{MIDI}}) = \left\lfloor \frac{\mathsf{f}_{\mathrm{clk}}}{2\mathsf{f}_{\mathrm{note}}(\mathsf{K}_{\mathrm{MIDI}})} \right\rfloor \tag{3}$$

For the lowest frequency, the largest counter value is needed. To optimally cover the whole counter range, the largest counter value is chosen as a power of two. The exponent resulting in the largest countervalue is defined as the Bitwidth of the counter. This value is chosen to be

$$\mathsf{BW} = 16\,\mathrm{bit}$$

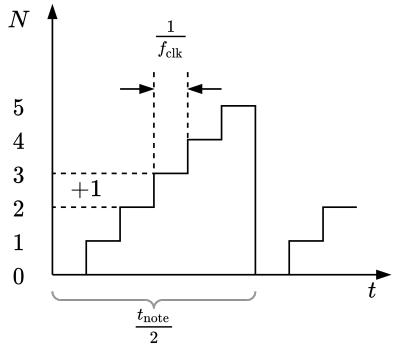

Figure 14: Generating custom time periods with a fixed clock cycle by defining a maximum countervalue

With the lowest piano note being an $\mathsf{A}_0 \equiv 27.5\,\mathrm{Hz}$ with the note number $\mathsf{K} = 21$, the required clock frequency is calculated by

$$\mathsf{f}_{\mathrm{clk}} = 2\mathsf{f}_{\mathsf{A}_0}\mathsf{N}_{\mathrm{max}} = 2 \cdot 27.5\,\mathrm{Hz} \cdot 2^{16} \approx 3.604\,\mathrm{MHz}$$

[3] Only half the countervalue is needed, because the squarewave has a duty cycle of 50 %. This allows to achieve one additional bit of resolution.

To mitigate counter overflows, the clock frequency is reduced to a round value of

$$f_{clk} = 3.5\,\text{MHz}, \quad t_{clk} = \frac{1}{f_{clk}} \approx 285.7\,\text{ns} \tag{4}$$

6.1.2 Precalculating the Octave

As stated, only the counter values of a single octave will be stored and are shifted left or right to double or half the frequency. With this approach, a common ground is found between calculating the counter value from the note number directly and looking it up from a LUT. The task is now to decide on one of the $\lfloor 127/12 \rfloor = 10$ possible octaves in the MIDI range. The options are:

- Choosing the highest octave and shift up. The highest frequencies require the smallest counter values, so this option would be the most memory efficient. But it has to be analysed, if the frequencies are precise enough not to propagate a significant error down to the lower frequencies.
- Choosing the lowest octave and shift down. This requires the most amount of space but eliminates the systematic error.
- Choosing a middle octave and shift up or down, if neither of the options above are optimal.

A deviation from the optimal note frequency occurs, because the calculated counter period in Equation 3 can only assume integer values while the decimal places are discarded. Higher frequencies are closer to the smallest time period to be resolved (Equation 4), which results in a higher relative error. Shifting from a smaller counter period upwards, ignores the lower significant bits (LSB), which could otherwise be used to store more precise values. As shifting is multiplicative, the same relative error applies to the resulting counter period directly. E.g. shifting $N_{CNT} = 165_d \equiv 10101010_b$ by four octaves yields:

$$10101010 << 4 = 101010100000 \quad \text{(introduced zeros)}$$

First, the highest MIDI octave is evaluated with Equation 3 and Equation 1 using Python[4]. The results are shown in Table 8. For the synthesized f_{synth}, first the equivalent counter value is calculated, floored, and then re-evaluated back to the frequency:

$$f_{synth}(K_{MIDI}) = \frac{f_{clk}}{N(K_{MIDI})} \cdot \frac{1}{2} \tag{5}$$

Shifting the count values from Table 8 eight octaves down to the lowest piano octave (multiplying N_{CNT} by $2^8 = 256$ and applying Equation 3), shows, that the relative error is preserved, with the results in Table 9. Here the synthesized frequency is calculated by

$$f_{synth}(K_{MIDI}) = \frac{f_{clk}}{2^8 \cdot N(K_{MIDI} + 12 \cdot 8)} \cdot \frac{1}{2} \tag{6}$$

If f_{synth} for the lower frequency range is directly calculated in the same way as for the values in Table 10 using Equation 5, it can be seen by the results in Table 10, that the relative error is reduced by factors up to 500.

[4] https://colab.research.google.com/drive/112KU6qkG-dn52oH6FUdRSUOJgx0EZzzK?usp=sharing

Table 8: Half counter values of the highest octave (8) by direct calculation from the MIDI note number. The error is caused by the truncation of the counter period to an integer value.

Note SPN	K_{MIDI} #	N_{CNT} #	f_{note} Hz	f_{synth} Hz	abs. Error Hz	rel. Error ‰
A_8	117	248	7040.00	7056.45	16.45	2.34
$A\sharp_8$	118	234	7458.62	7478.63	20.01	2.68
B_8	119	221	7902.13	7918.55	16.41	2.08
C_9	120	209	8372.02	8373.20	1.19	0.14
$C\sharp_9$	121	197	8869.84	8883.24	13.40	1.51
D_9	122	186	9397.27	9408.60	11.32	1.21
$D\sharp_9$	123	175	9956.06	10000.00	43.93	4.41
E_9	124	165	10548.08	10606.06	57.97	5.50
F_9	125	156	11175.30	11217.94	42.64	3.82
$F\sharp_9$	126	147	11839.82	11904.76	64.94	5.48
G_9	127	139	12543.85	12589.92	46.07	3.67
$G\sharp_9$	128	131	13289.75	13358.77	69.02	5.19

Table 9: Half counter values of the lowest octave (0) by using the counter values from the eighth octave and logically shifting the counter period upwards. The error is caused by the shift operation omitting the LSB values.

Note SPN	K_{MIDI} #	N_{CNT} #	f_{note} Hz	f_{synth} Hz	abs. Error Hz	rel. Error ‰
A_0	21	63488	27.50	27.56	0.06	2.34
$A\sharp_0$	22	59904	29.14	29.21	0.08	2.68
B_0	23	56576	30.87	30.93	0.06	2.08
C_1	24	53504	32.70	32.71	0.00	0.14
$C\sharp_1$	25	50432	34.65	34.70	0.05	1.51
D_1	26	47616	36.71	36.75	0.04	1.21
$D\sharp_1$	27	44800	38.89	39.06	0.17	4.41
E_1	28	42240	41.20	41.43	0.23	5.50
F_1	29	39936	43.65	43.82	0.17	3.82
$F\sharp_1$	30	37632	46.25	46.50	0.25	5.48
G_1	31	35584	49.00	49.18	0.18	3.67
$G\sharp_1$	32	33536	51.91	52.18	0.27	5.19

Table 10: Half counter values of the lowest octave (0) by direct calculation from the MIDI note number. The error almost negligible, because the LSBs are used to determine the counter period up to the resolution of the clock period.

Note SPN	K_{MIDI} #	N_{CNT} #	f_{note} Hz	f_{synth} Hz	abs. Error mHz	rel. Error ‰
A_0	21	63636	27.5000	27.5002	0.2	0.01
$A\sharp_0$	22	60064	29.1352	29.1356	0.4	0.01
B_0	23	56693	30.8677	30.8680	0.3	0.01
C_1	24	53511	32.7032	32.7036	0.4	0.01
$C\sharp_1$	25	50508	34.6478	34.6480	0.1	0.00
D_1	26	47673	36.7081	36.7084	0.3	0.01
$D\sharp_1$	27	44997	38.8909	38.8915	0.6	0.02
E_1	28	42472	41.2034	41.2036	0.2	0.00
F_1	29	40088	43.6535	43.6540	0.4	0.01
$F\sharp_1$	30	37838	46.2493	46.2498	0.5	0.01
G_1	31	35714	48.9994	49.0004	1.0	0.02
$G\sharp_1$	32	33710	51.9131	51.9134	0.3	0.01

Conclusion Even though the error is larger on a relative scale, it has to be evaluated if the absolute frequency deviation is audible. Detecting such small deviations by ear is heavily dependent on the individual. The relative error can be applied to all the other octaves and tested, if there are audible differences. An analysis by the author was carried out in the DAW Ableton Live using the arbitrary wave generator *Operator*.

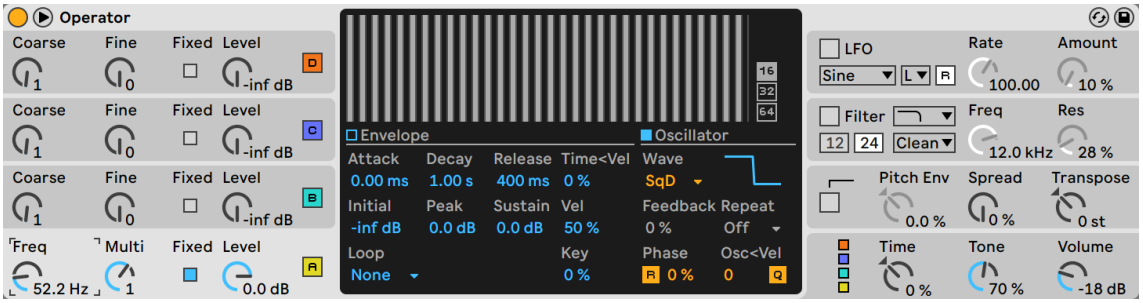

Figure 15: In the Operator, the waveform is selected as a digital squarewave (*SqD*). The frequnecy can be set precisely with the lower left *Freq* dial.

The conclusion of this experiment is that only deviations in the middle octaves can be just slightly perceived. Due to this minor influence on the instrument quality, the option with the least amount of required memory is chosen.

6.1.3 Table Optimizations

The candidates to be hardwired into the IC are now listed in Table 11. By analysing the bitpattern of the binary values, it can be seen, that the most significant bit is common to all values. Now, even one less bit needs to be stored for a total of 7 bit. The omitted bit is added again in the oscillators themselves. This optimization reduces routing costs, as now one less wire needs to be routed to each oscillator.

Table 11: Final values of the half counter periods to be stored in the LUT. The values are stored with the removal of redundant bits.

Decimal Values	Binary Values	Binary without redundancy	Decimal without redundancy
248	11111000	01111000	120
234	11101010	01101010	106
221	11011101	01011101	93
209	11010001	01010001	81
197	11000101	01000101	69
186	10111010	00111010	58
175	10101111	00101111	47
165	10100101	00100101	37
156	10011100	00011100	28
147	10010011	00010011	19
139	10001011	00001011	11
131	10000011	00000011	3

6.1.4 Implementation

The `note2cnt` module contains the lookup table and mostly combinatorial logic for indexing the LUT. The ports of the module are seen in Listing 16. The lookup table is declared as a register array, which is initially set to the predefined values in Table 11. The indexing starts with the A note. As the first A_0 note starts at 21, the value is first subtracted as shown in Listing 18.

Listing 16: Module ports of the `note2cnt` module

```
module note2cnt (
    input wire clk_i,
    input wire nrst_i,
    input wire [`MIDI_NOTE_BW-1:0] note_i,
    output reg [`OSC_ROM_BW-1:0] baseCntPeriod_o,
    output reg [3:0] shift_o
);
```

Listing 17: Implementation of the register, which stores the values of the LUT listed in Table 11

```
    reg [`OSC_ROM_BW-1:0] noteRom [11:0];
    initial begin
        noteRom[0] = 120;
        noteRom[1] = 106;
        noteRom[2] = 93;
        noteRom[3] = 81;
        noteRom[4] = 69;
        noteRom[5] = 58;
        noteRom[6] = 47;
        noteRom[7] = 37;
        noteRom[8] = 28;
        noteRom[9] = 19;
        noteRom[10] = 11;
        noteRom[11] = 3;
    end
```

Listing 18: Subtract the base note from the note present on the input, so that the new numbering of the notes starts at 0. If the incoming note is somehow below the lowest piano note, the note is passed directly to prevent negative values.

```
    always @(*) begin
        if (note_i < 21) begin
            actualNote = `OSC_ROM_BW'd0;
        end else begin
            actualNote = note_i - `OSC_ROM_BW'd21;
        end
    end
```

To index the LUT, it needs to be extracted which of the twelve notes in the octave is present at the input. To do this, one would classical use the modulo (%) operator. However, using it proved to instantly require more space. What also needs to be known, is the octave number to determine the amount to shift the counter register. For this, the note number has to be divided by 12, which also leads to the issue, that divisions are costly to implement in hardware. For this the workarounds are shown in Listing 19 and Listing 20.

Listing 19: Determine the shift amount by applying thresholds of note range with a if-else structure, to work around the division.

```
    always @(*) begin
        // shift = actualNote / 12
        if (actualNote < 12) octave = 4'h0;
        else if (actualNote < 24) octave = 4'h1;
        else if (actualNote < 36) octave = 4'h2;
        else if (actualNote < 48) octave = 4'h3;
        else if (actualNote < 60) octave = 4'h4;
        else if (actualNote < 72) octave = 4'h5;
        else if (actualNote < 84) octave = 4'h6;
        else if (actualNote < 96) octave = 4'h7;
        else octave = 4'h8;
    end
```

Listing 20: Calculate the note index by subtracting the octave base.

```
1     always @(*) begin
2         // noteIndex = actualNote % 12
3         noteIndex = actualNote - octave * 12;
4         baseNoteCnt = noteRom[noteIndex[3:0]];
5     end
```

At last, the time base value and the shift amount are registered to the output of the module in Listing 21. It has to be noted, that Listing 19 and Listing 20 infer a long combinatorial chain from the input to the register. Due to the propagation delay of the combinatorics, it is not guaranteed, that the output is present within one clock period. This is dependent on the gate implementations of the PDK.

Listing 21: Assigning the time base and the shift value. As the counter value needs to be shifted down *to* the octave, the shift value is the maximum octave minus the current octave.

```
1     always @(posedge clk_i or negedge nrst_i) begin
2         if(!nrst_i) begin
3             baseCntPeriod_o <= {(`OSC_ROM_BW){1'b0}};
4             shift_o <= 4'h0;
5         end else begin
6             baseCntPeriod_o <= baseNoteCnt;
7             shift_o <= 4'h8 - octave;
8         end
9     end
```

6.2 Oscillator

Each oscillator in the oscillator stack now receives the channel number and control strobes from the MIDI decoder and the time base and shift value from the LUT logic. The inputs of the oscillator only take effect, if the MIDI channel match, the ID generated with the *generate-for* (compare Listing 15, line 13).

The oscillator uses combinatorial signal to control its state:

- `start`: High, if the channels match and an *on* strobe is received.
- `stop`: High, if the channels match and an *off* strobe is received. An additional contition is, that the counter value at the input must match the currently playing note. This is needed, that if a second note overrides the currently playing note, releasing the old key does not turn off the oscillator. Only releasing the key playing the note should turn off the output.
- `cntReached`: The free running counter is shifted right by the shift amount and compared with the timebase. If the signals match, this signal is high. Also, the previously omitted bit from the LUT is added in this comparison.
- `enabled`: While start and stop are strobe signals, this signal is high all the time while the oscillator is playing.

When the oscillator starts, the time base and the shift values are latched. With the oscillator enabled, whenever the counter reaches its limit, the counter is reset and the output wave is toggled with an XOR operation.

The oscillator module was optimized to route as few wide signals as possible and reducing the size for logic operation.

7. Encoding the number of active oscillators

A feature of the design is, that the pin `uo[7]` outputs a PWM signal, which encodes the number of actively playing oscillator. The idea behind this, is that if the PWM signal is heavily lowpassed, the average DC-voltage can be used to control the gain of external mixing circuitry.

The synth module registers, which oscillators are currently active in a register. The register stores a `1` at the index of the associated channel if the oscillator is active and `0` otherwise. To convert this logic into a PWM signal, the number of high bits are converted into a number and then fed into a PWM module which is based on a counter.

Counting the bits in the active oscillator register is straight forward by summing each bit together shown in Listing 22.

Listing 22: A for loop generates an adder circuit for each bit in the input register.

```
module bitcount #(
    parameter WORDLEN = 7
)(
    input wire [WORDLEN-1:0] word_i,
    output reg [$clog2(WORDLEN+1)-1:0] count_o
);
    localparam CNT_BW = $clog2(WORDLEN+1);
    integer i;
    always @(*) begin
        count_o = 0;
        for (i = 0; i < WORDLEN; i = i + 1) begin
            count_o = count_o + {{(CNT_BW - 1) {1'b0}}, word_i[i]};
        end
    end
endmodule // bitcount
```

The resulting count of the active oscillator register, determines the duty cycle of the PWM. The maximum number of voices is set as the total counter period of the PWM. A trivial implementaton of a PWM generator is given in Listing 23.

Listing 23: The output is high, until a counter reaches the duty cycle count, which must be less than the period count. The output is low for the remaining period count. The counter gets a synchronous reset, if the period count is reached.

```
module pwm #(
    parameter PWM_BW = 3
) (
```

```
4       input wire clk_i,
5       input wire nrst_i,
6       input wire [PWM_BW-1:0] onCnt_i,
7       input wire [PWM_BW-1:0] periodCnt_i,
8       output wire pwm_o
9   );
10
11      wire cntNSyncRst;
12      wire [PWM_BW-1:0] pwmCountVal;
13
14      counter #(
15          .BW(PWM_BW)
16      ) pwmCounter_inst (
17          .clk_i(clk_i),
18          .nrst_i(nrst_i),
19          .nrstSync_i(cntNSyncRst),
20          .count_o(pwmCountVal)
21      );
22
23      assign cntNSyncRst = (pwmCountVal < periodCnt_i);
24      assign pwm_o = (pwmCountVal < onCnt_i);
25
26  endmodule // pwm
```

8. System Testing

Edge Cases

Limitations. Which tests are omitted. Correctness of the midi Input Bytoe Order for example.

CocoTB Testresults, Design Simulation+validation, Measurements, Waveforms

9. External Hardware

9.1 MIDI-Source

As the device does not fully comply to the GM-1 specs, it is best to provide MIDI inputs from a controlled environment. Pluggin in a full-fledged midi controller directly might lead to unwanted behaviour.

Example of a USB-MIDI controller.

In Principle Direct connection is Possible but a Powersupply is required.

Setting up a Virtual MIDI-Source with a DAW, Hairless MIDI and

PMOD as a PHY for the native Differential midi connection.

9.2 Input Side

Digital Frontend converting simultaneous Notes into individual Channel

9.3 Output Side

Audio Mixing Circuit, Gain Control, Speaker Driver

Option for mild subtractive Sound-Design

- Squarewaves provide all odd harmonics of the fundamental frequencies, which can be filtered by an external system to create different Sounddesigns

- optimal Would be a Sawtooth as it contains all harmonics but violated the digitality of the ASIC Output.

Appendix A. Used Software

Table 12: List of used software for development and comissioning this project

Name	Version	Description	Usage
Ableton Live 11 Suite	11.3.43	DAW	Spectral analysis for instruments. Experiments for audible perceptions of frequency deviations.

appendix: Setting up a development env.

Appendix B. Wikipedia References

W MIDI - Wikipedia

W MIDI tuning standard - Wikipedia

W Scientific pitch notation - Wikipedia

W 12 equal temperament - Wikipedia

W A440 (pitch standard) - Wikipedia

References

[1] Ableton AG. Live Grand Piano. https://www.ableton.com/en/packs/grand-piano. Accessed: 2026-01-02.

[2] The MIDI Manufacturers Association. General MIDI System Level 1 Specification. https://midi.org/general-midi-level-1, 1996. Accessed: 2025-12-25.

[3] The MIDI Manufacturers Association. MIDI 1.0 Detailed Specification. https://midi.org/midi-1-0-detailed-specification, 1996. Accessed: 2026-01-13.

[4] The MIDI Manufacturers Association. General MIDI System Level 2 Specification. https://midi.org/general-midi-level-2-2, 2007. Accessed: 2025-12-25.

[5] The MIDI Manufacturers Association. Summary of MIDI 1.0 Messages. https://midi.org/summary-of-midi-1-0-messages, 2020. Accessed: 2025-12-25.

[6] Weisstein, Eric W. Fourier Series–Square Wave. From MathWorld–A Wolfram Resource. https://mathworld.wolfram.com/FourierSeriesSquareWave.html. Accessed: 2026-01-02.